

API Port de Plaisance Russell



DEMO RENDER

VISIT

Application **Node.js + Express + MongoDB** pour la gestion :

- des **catways** (emplacements bateaux),
- des **réservations**,
- des **utilisateurs** (avec rôles **admin** / **user**),
- avec une **interface HTML/EJS** intégrée **et** une **documentation Swagger**.

L'authentification se fait via **JWT** (stocké dans un cookie **httpOnly** ou dans l'en-tête **Authorization: Bearer <token>**).



Versions disponibles

- **Version Développement (publique – ce repo)**
 - Exécution locale : `http://localhost:3000`
 - Sert de support pédagogique, sans secrets sensibles dans le code. → Documentation du code avec JSdoc
 - **Version Production (privée – déployée)**
 - Déployée automatiquement sur **Render**
 - URL : `https://devoir-api-port-de-plaisance-russell-prod.onrender.com`
 - Accessible avec des comptes **user** et **admin** fournis pour les tests. → Pas documentation du code avec JSdoc pour ne pas alourdir le déploiement et garder la prod légère.
-



Installation & Lancement

1. Cloner le projet

```
git clone https://github.com/CL4P-TP-afk/Devoir_API_Port_de_Plaisance_Russell_Dev
```

2. Installer les dépendances

- ⚠ vérifier que vous êtes bien à la racine du projet !

```
npm install
```

3. Configurer les variables d'environnement

Créer un fichier env/.env (ou adapter ton .env si tu utilises env-cmd) :

```
env
PORT=3000
MONGO_URI=mongodb+srv://<user>:<password>@<cluster>/<db>?
retryWrites=true&w=majority
MONGO_DBNAME=apinode
SECRET_KEY=un_secret_tres_long_et_complexe
```

4. Importation des jeux de données initiales

- ⚠ Réinitialise complètement la base et recharge le jeu de données immuable seed-init/ :

```
npm run seed:init
```

⚠ Après npm run seed:init, tous les utilisateurs ont le mot de passe: ChangeMe123! (modifiable dans scripts/seed-init.js)

5. Lancer l'application

```
npm start
```

⚠ Nécessite une base MongoDB locale ou distante.

📁 Structure du projet

```
.
├── app.js                # Point d'entrée Express
├── bin/www               # Lanceur serveur
├── db/mongo.js           # Connexion MongoDB
├── routes/               # Définition des routes (API + vues HTML)
│   ├── auth.js
│   ├── catways.js
│   ├── reservations.js
│   ├── users.js
│   ├── dashboard.js
│   ├── reserver.js
│   └── docs.js
├── models/               # Schémas Mongoose (User, Catway, Reservation)
├── services/             # Logique métier (validation, règles)
├── middlewares/          # Authentification & rôles
├── views/                # Templates EJS (UI)
├── swagger.yaml          # Documentation OpenAPI 3
├── swagger.js            # Branche Swagger sur /api-docs
└── public/               # Fichiers statiques (CSS/JS)
```

Authentification & Rôles

JWT signé avec **SECRET_KEY**.

- Stocké dans cookie **httpOnly** ou envoyé dans **Authorization: Bearer** .
- Rôles :
 1. admin : accès complet (gestion utilisateurs, catways, réservations).
 2. user : accès limité (consultation et création de réservations).

Documentation API

- Version intégrée : <http://localhost:3000/docs> (affiche Swagger dans une page EJS avec bouton retour vers l'UI).
- Version Swagger UI brute : <http://localhost:3000/api-docs> (pleine page Swagger, directement générée depuis swagger.yaml).

Endpoints principaux

1. Auth

- POST /auth/login → connexion (JWT + cookie, redirection vers dashboard).
- GET /auth/logout → déconnexion (suppression cookie, retour à l'accueil).

2. Catways

- GET /catways → liste des catways (HTML).
- POST /catways → créer un catway.
- PUT /catways/{id} → modifier état/réservabilité.
- DELETE /catways/{id} → supprimer un catway.

3. Réservations

- GET /catways/{id}/reservations → liste des réservations d'un catway.
- POST /catways/{id}/reservations → créer une réservation.
- PUT /catways/{id}/reservations/{idReservation} → modifier une réservation.
- DELETE /catways/{id}/reservations/{idReservation} → supprimer une réservation.

4. Utilisateurs

- GET /users → liste/gestion des utilisateurs (HTML).
- POST /users → créer un utilisateur.

- PUT /users/{id} → mettre à jour un utilisateur.
- DELETE /users/{id} → supprimer un utilisateur.

Règles métier principales

1. Catways

- catwayNumber unique et ≥ 1 .
- catwayType $\in \{\text{long}, \text{short}\}$.
- isReservable = true/false (par défaut true).

2. Réservations

- clientName doit exister dans User.name.
- Refus si catway.isReservable === false.
- Refus si chevauchement de dates pour un même catway.
- Dates : format date ou date-time.

3. Utilisateurs

- name ≥ 3 caractères.
- email valide et unique.
- password fort (minuscule + majuscule + chiffre).
- role $\in \{\text{admin}, \text{user}\}$.
- password hashé avec bcrypt.

Scénario de test rapide

- Créer un compte admin (/users via interface ou seed).
- Connexion : /auth/login → cookie JWT.
- Dashboard : /dashboard → navigation vers menus.
- Créer un catway : /catways (ex. n°1, type short).
- Créer une réservation : /catways/{id}/reservations.
- Vérifier qu'un chevauchement de dates est refusé.
- Consulter la doc : /docs ou /api-docs.

Commandes utiles

Lancer avec env-cmd (env-cmd -f ./env/.env nodemon ./bin/www) :

```
env-cmd -f ./env/.env nodemon ./bin/www
```

Lancer en dev avec nodemon (env-cmd -f ./env/.env.dev nodemon ./bin/www) :

```
npm run dev
```

Lancer en prod avec nodemon (env-cmd -f ./env/.env.prod nodemon ./bin/www) :

```
npm run prod
```

Import/export de données :

- importer sans rien effacer (ajoute/ignore doublons) :

```
npm run seed
```

- importer en purgeant (⚠ → reset complet) :

```
npm run seed:reset
```

- Importer le jeu de secours immuable seed-init/ :

```
npm run seed:init
```

- Sauvegarder l'état actuel de la base vers JSON (⚠ → écrase seed/*.json avec les données en DB) :

```
npm run dump
```

Notes sécurité

- **JWT** stocké en cookie **httpOnly** → limite les risques **XSS**.
- Vérification stricte des rôles (isAdmin) sur routes sensibles.
- Validation de toutes les entrées côté serveur (Mongoose + services).

- Mot de passe jamais stocké en clair (toujours hashé).

Documentation du code (JSDoc)

La documentation interne du code est générée automatiquement grâce à JSDoc.

 Générer la doc :

```
npm run docs:jsdoc
```

- Utilise la configuration jsdoc.json à la racine.
- La sortie est générée dans : ./docs/jsdoc

 Consulter la doc en local :

```
npm run docs:serve
```

- Un petit serveur Express sert la doc sur : http://localhost:3001/dev-docs
- Le fichier d'entrée côté disque est : ./docs/jsdoc/index.html

 Notes :

- Les commentaires JSDoc couvrent routes, services, middlewares et models.
- Les endpoints HTML (vues EJS) et les redirections sont documentés (types, effets, messages).
- La doc JSDoc est destinée au développement ; elle n'est pas publiée sur l'environnement Production (Render).

Bonus – Extension “Espace Utilisateur”

Dans un souci d'amélioration et d'évolution du projet, j'ai pris l'initiative d'ajouter une ouverture contrôlée de l'API aux utilisateurs du port via un rôle user.

Objectif

Permettre aux clients du port (non administrateurs) de se connecter à l'application grâce à un compte user. Ce rôle n'était pas demandé dans le cahier des charges, mais il prépare une extension naturelle du système.

Évolutions envisagées

Dans un cadre d'amélioration future, cette ouverture pourrait permettre :

- la consultation et la modification du profil utilisateur ;
- la création ou modification de demandes de réservation en ligne ;
- une gestion personnelle de l'historique des réservations.

État actuel

- Le rôle user existe dans la base et peut se connecter.
- L'espace correspondant (/user-dashboard) affiche actuellement une page d'attente indiquant que la fonctionnalité est en construction.
- Les droits précis (lecture/écriture sur certaines routes) restent à définir dans une future version.